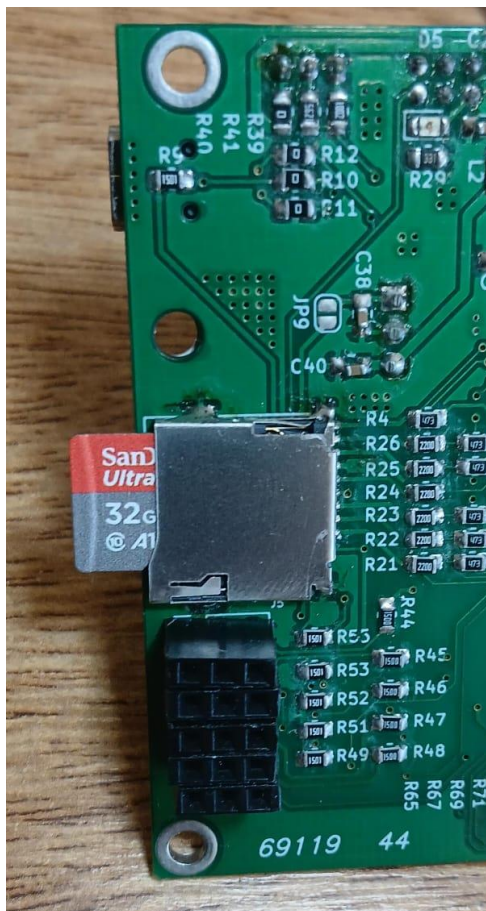
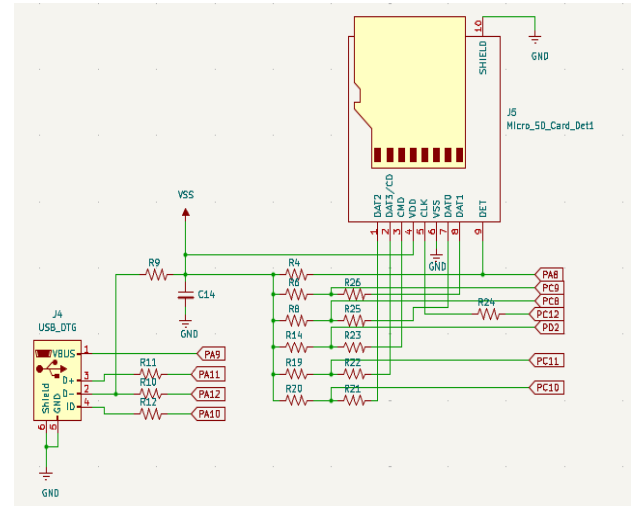


MicroSD

en modo SDMMC de 4 bits

Será una pieza clave para el sistema, tanto para el desarrollo como para la operación real.
Entre las funciones principales estará el registro de Loggings , guardando continuamente datos como

timestamp
lat, lon (GPS)
velocidad (GPS o estimada)
aceleraciones (IMU)
velocidades angulares (gyro)
actitud (roll, pitch, yaw)
estado del EKF
error de navegación
salidas del controlador (PWM motor)
estado de misión



- esto nos permite analizar fallos, ajustar el EKF, ajustar controladores PID, validar estabilidad.

- Además lo usaremos para almacenar los Waypoints o misiones con el formato,

WP1: lat, lon

WP2: lat, lon

WP3: lat, lon

...

y lo podemos cargar al inicio con la ventaja de poder cambiar la misión sin reprogramar el micro, ideal para pruebas de campo

```
f_open("mission.txt")
```

```
f_read(...)
```

- también nos permite guardar configuración del sistema de manera editable, esto es muy importante en el desarrollo porque se puede ajustar sin recompilar y se puede tener distintos perfiles.

- se puede usar también como buffer de telemetría por si se pierde la comunicación, se guarda todo en SD, lo recupero después , muy útil en navegación real.

- al final se puede hacer un análisis de procesamiento con los datos obtenidos.

Los pines del STM32H7 involucrados son ,

PC8 -> SDMMC1_D0
PC9 -> SDMMC1_D1
PC10 -> SDMMC1_D2
PC11 -> SDMMC1_D3
PC12 -> SDMMC1_CK
PD2 -> SDMMC1_CMD
PA8 -> Card Detect,

Flujo de acceso a microSD mediante FatFs + SDMMC



necesito agregar 4 bloques, que son ,

- . driver HAL SDMMC1
- . pines GPIO SDMMC1

- . FatFs
- . Funciones para crear, escribir , leer y cerrar archivos.

FatFs está configurado solo para formato 8.3, es decir, archivos de hasta 8 caracteres de nombre y 3 de extensión. Si algún archivo no cumple con estas características da un error de código número 6 que corresponde a FR_INVALID_NAME.

Integración del USB MSC + MicroSD

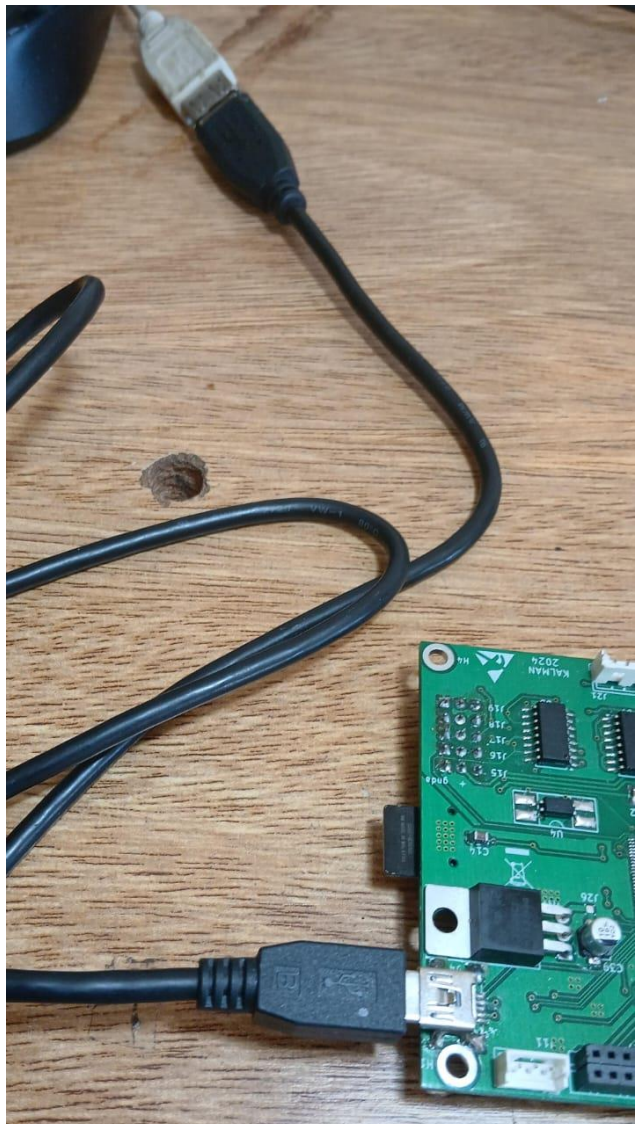
El sistema implementa una interfaz USB tipo Mass Storage Class (MSC) utilizando el periférico USB OTG FS del STM32H743 y una tarjeta microSD conectada mediante SDMMC1.

El objetivo principal es permitir:

- extracción de logs de navegación
- descarga de datos de sensores
- análisis de telemetría
- recuperación de archivos de configuración
- mantenimiento del sistema

La tarjeta microSD es utilizada por el autopiloto durante el funcionamiento normal para almacenar información de navegación, sensores y eventos.

Cuando se requiere acceder a los archivos desde una PC, el STM32 entra en modo USB MSC y expone la tarjeta SD como un disco externo.



Modo de funcionamiento

tenemos dos modos principales, autopiloto y USB como se ve en la tabla,

Modo AUTOPILOTO	Modo USB MSC
Características: • ejecución del algoritmo de control • lectura GPS • lectura IMU • lectura barómetro • manejo PWM motores • navegación • logging en tarjeta microSD mediante FATFS	Características: • el STM32 expone la microSD como unidad USB • Linux/Windows detecta la SD como disco externo • se pueden leer, crear y modificar archivos desde la PC
En este modo: • USB MSC NO está activo • la tarjeta SD es utilizada exclusivamente por el STM32	En este modo: • el autopiloto NO se ejecuta • FATFS no se monta desde el STM32 • el STM32 no debe escribir en la SD

Secuencia correcta para acceder a la microSD desde la PC

Paso 1

Apagar el sistema.

Paso 2

Conectar el cable USB entre la PC y el STM32.

Paso 3

Encender el sistema.

Paso 4

El STM32 detecta VBUS mediante PA9 y entra automáticamente en modo USB MSC.

Paso 5

La PC detecta la tarjeta microSD como disco externo.

Ejemplo Linux:

```
lsblk
```

Ejemplo:

```
sdb          8:16    1  29,7G  0 disk
└─sdb1      8:17    1  29,7G  0 part /media/usuario/AUTOPILOTO
```

Paso 6

Realizar lectura/escritura de archivos normalmente.

Paso 7

Expulsar correctamente la unidad desde el sistema operativo.

Paso 8

Desconectar USB.

Paso 9

Reiniciar el sistema.

Al reiniciar sin USB conectado:

- el STM32 vuelve automáticamente al modo AUTOPILOTO

Detección automática USB mediante PA9

Se utiliza el pin PA9 como detector de VBUS.

Configuración GPIO:

```
GPIO_InitStruct.Pin = GPIO_PIN_9;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_PULLDOWN;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
```

Función utilizada:

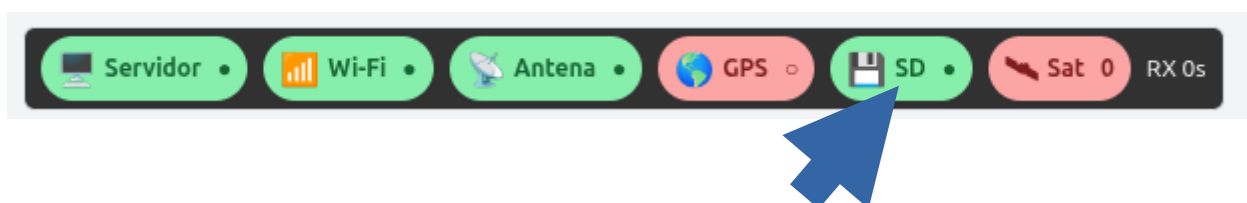
```
uint8_t USB_Conectado(void)
{
    return (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9) == GPIO_PIN_SET);
}
```

Funcionamiento:

- USB desconectado → PA9 = 0
- USB conectado → PA9 = 1

El uso de PULLDOWN interno fue necesario debido a que PA9 quedaba flotando eléctricamente cuando no existía conexión USB.

En la interfase podemos ver el estado de conexión de la memoria SD con un icono en la barra de estado de sensores de la sección **PANEL DE CONTROL**.



Si no está conectada la memoria SD, el ícono aparece con fondo de color rojo

Configuración SDMMC1

Archivo:

```
Core/Src/main.c
```

Función:

```
MX_SDMMC1_SD_Init()
```

Configuración estable utilizada:

```
hsd1.Init.BusWide = SDMMC_BUS_WIDE_4B;  
hsd1.Init.ClockDiv = 8;
```

Observaciones:

- modo 4 bits mejora significativamente la velocidad
- ClockDiv=8 entregó funcionamiento estable
- configuraciones demasiado rápidas pueden provocar:
 - desaparición del dispositivo USB
 - cuelgue del sistema
 - pérdida de comunicación SD

Configuración USB

Archivo principal:

```
USB_DEVICE/Target/usbd_conf.c
```

Configuración importante:

```
hpcd_USB_OTG_FS.Init.vbus_sensing_enable = ENABLE;
```

Observación:

Con VBUS sensing habilitado el hardware USB puede no enumerar correctamente dependiendo del diseño de la placa.

Se habilito el VBUS sensing interno y se implementó detección manual mediante PA9.

```
uint8_t USB_Conectado(void)
```

```
{  
    return (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9) == GPIO_PIN_SET);  
}
```

Archivos principales involucrados

USB

USB_DEVICE/App/usb_device.c
USB_DEVICE/App/usbd_storage_if.c
USB_DEVICE/Target/usbd_conf.c

SDMMC

Core/Src/main.c

FATFS

FATFS/App/fatfs.c
Middlewares/Third_Party/FatFs/

Main

Core/Src/main.c

Recomendaciones importantes

- NO utilizar FATFS y USB MSC simultáneamente

Nunca:

- escribir desde STM32
- mientras la PC está accediendo a la SD

Esto puede corromper el sistema de archivos.

- Siempre expulsar correctamente la unidad USB

Antes de desconectar USB:

- desmontar unidad desde Linux/Windows

- Mantener el modo USB separado del modo AUTOPILOTO

Arquitectura recomendada:

USB conectado al arranque → modo MSC
USB desconectado → autopiloto

En qué casos se usa la memoria microSD

logs - registros secuenciales de datos o eventos generados durante el funcionamiento. Por ejemplo hora, latitud, longitud, rumbo, velocidad, altura. O también puede ser error GPS, waypoint alcanzado, batería baja.

El uso típico es ,

análisis posterior de navegación

debug del sistema

reconstrucción de trayectoria

formato de archivo típico,

log_001.txt

vuelo_2026_05_14.csv

telemetría – información transmitida en tiempo real hacia la estación terrena, por ejemplo,

posición GPS

estado EKF

roll/pitch/yaw

nivel batería

estado motores

históricos -

Es decir, en la telemetría se almacenan datos en vivo

históricos – son datos almacenados durante largo periodo de tiempo para su posterior análisis.

Como por ejemplo,

consumo promedio

errores frecuentes

rendimiento de navegación

mapa de recorridos

Es decir, puede representar información acumulada

configuración editable no crítica – Parámetros que el usuario puede modificar sin comprometer el arranque básico del sistema, como por ejemplo,

ganancias PID

velocidad crucero

distancia entre waypoints

frecuencia de telemetría

modo navegación

nombres de archivos típicos son,

config.txt

config.json

backup de calibración – copias de seguridad de parámetros de calibración guardados en la memoria flash.

Puede llamarse, backup_calib.txt

las ventajas pueden ser,

- restaurar calibración
- copiar calibración entre placas
- analizar valores desde PC

Waypoints para autopiloto

Los waypoints van a estar almacenados en un archivo en la memoria SD que se llama PLANNING.CSV para poder tener acceso fácil con las funciones `f_gets()` y `sscanf()`.

El formato que va a tener la información es,

```
# TYPE,ID,LAT,LON,ALT,VEL,RADIUS
BASE,Base,-34.58000000,-58.38000000,0.00,0.00,0.00
WP,WP2,-34.56732103,-58.33431244,0.00,2.00,5.00
WP,WP3,-34.57749037,-58.28487396,0.00,2.00,5.00
WP,WP4,-34.55489013,-58.25603485,0.00,2.00,5.00
WP,WP5,-34.56675603,-58.21483612,0.00,2.00,5.00
WP,WP6,-34.59076515,-58.20281982,0.00,2.00,5.00
WP,WP7,-34.60770859,-58.19114685,0.00,2.00,5.00
WP,WP8,-34.63198816,-58.20899963,0.00,2.00,5.00
WP,WP9,-34.64440752,-58.23543549,0.00,2.00,5.00
```

donde sus campos son,

TYPE = BASE o WP

ID = Base, WP1, WP2...

LAT = latitud decimal

LON = longitud decimal

ALT = altura en metros

VEL = velocidad deseada

RADIUS = radio de aceptación en metros

Estructura del waypoint:

```
typedef struct
{
    char id[16];
    double lat;
    double lon;
    float alt;
    float vel;
    float radio_m;
} Waypoint_t;
```

LÓGICA DE NAVEGAR HACIA UN WAYPOINT,

```
Waypoint_t *wp = WP_GetCurrent();

if (wp != NULL)
{
    navegar_hacia(wp->lat, wp->lon, wp->alt, wp->vel, wp->radio_m );

    if (WP_CheckReached(lat_actual, lon_actual, alt_actual))
    {
        WP_Advance();
    }
}
```

Registro de Navegación y Análisis de Trayectorias

Con el objetivo de facilitar las pruebas de campo y el análisis posterior del comportamiento del vehículo, se incorporó en la interfaz gráfica un sistema de registro de recorridos basado en archivos CSV almacenados automáticamente en la memoria microSD.



La función se activa mediante el botón "**Guardar Recorrido**", ubicado en la sección de control de navegación. Cuando el operador activa esta opción, el sistema comienza a registrar periódicamente información de navegación en un archivo CSV independiente. Cada nueva sesión genera automáticamente un nuevo archivo con nombre correlativo, evitando sobrescribir registros anteriores.

Por ejemplo:

GPSLOG01.CSV

GPSLOG02.CSV

GPSLOG03.CSV

...

Cada archivo contiene información cronológica de la navegación, incluyendo:

timestamp

estado GPS

fix GPS

latitud

longitud

altitud

velocidad sobre el terreno (SOG)

rumbo GPS (COG)

rumbo estimado por el EKF

cantidad de satélites

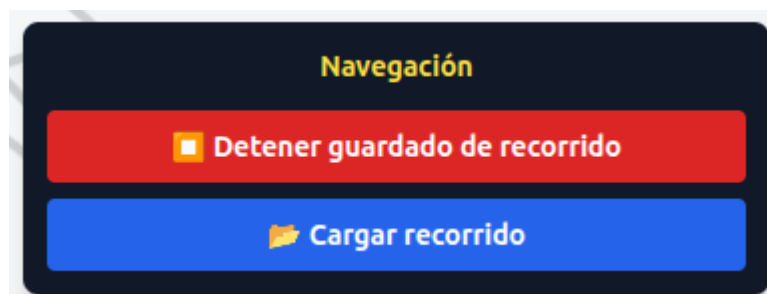
HDOP

EHPE

EVPE

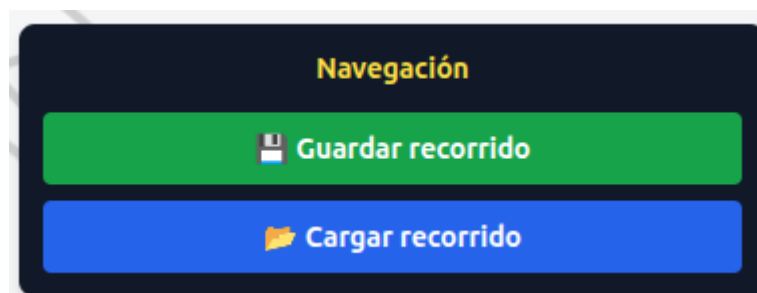
El intervalo de almacenamiento puede configurarse según las necesidades del ensayo. Durante las pruebas realizadas se utilizó un período de 30 segundos entre registros.

Una vez activado el almacenamiento, el sistema continúa registrando datos hasta que el operador vuelve a presionar el mismo botón, momento en el cual el registro se detiene y el archivo queda almacenado permanentemente en la memoria microSD para su posterior análisis.

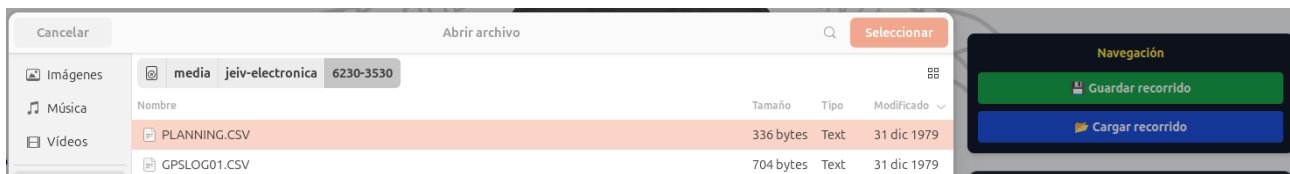


Representación de recorridos sobre mapas

Los archivos CSV generados pueden ser posteriormente cargados en la interfaz gráfica mediante el botón "**Cargar Recorrido**".



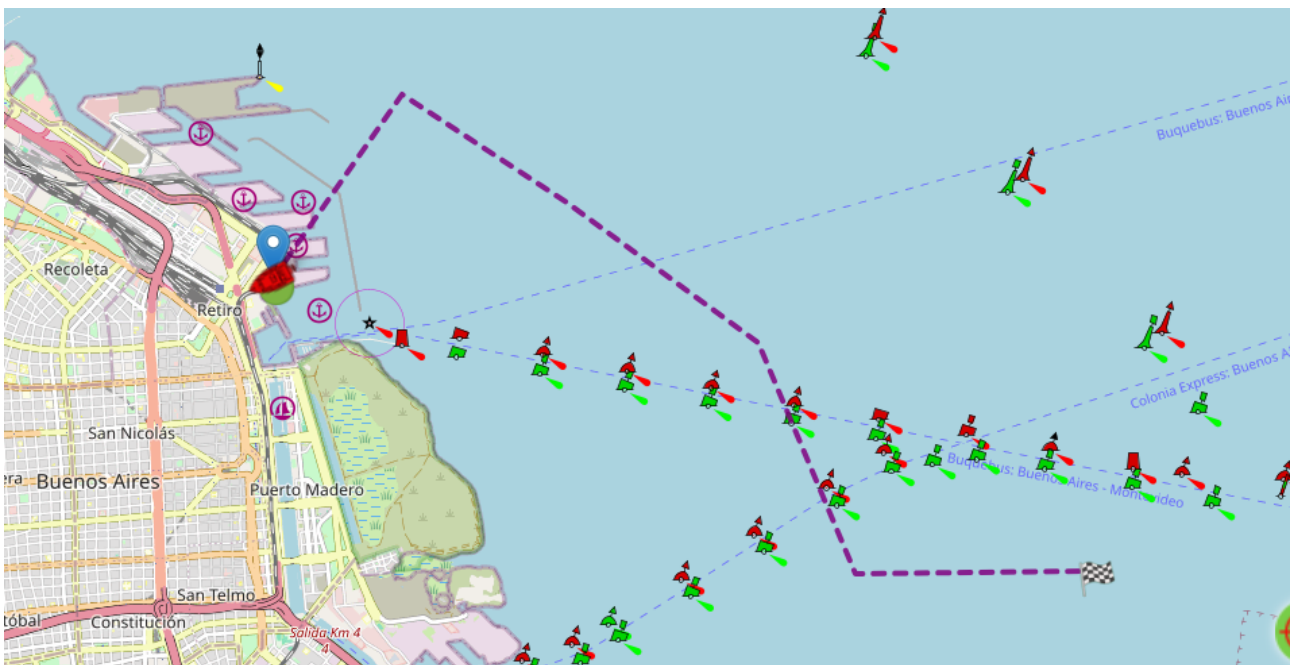
Esta función permite seleccionar cualquiera de los archivos GPSLOG generados anteriormente y representar gráficamente las coordenadas registradas sobre el mapa interactivo implementado en el componente MapView.jsx.



De esta forma es posible visualizar:

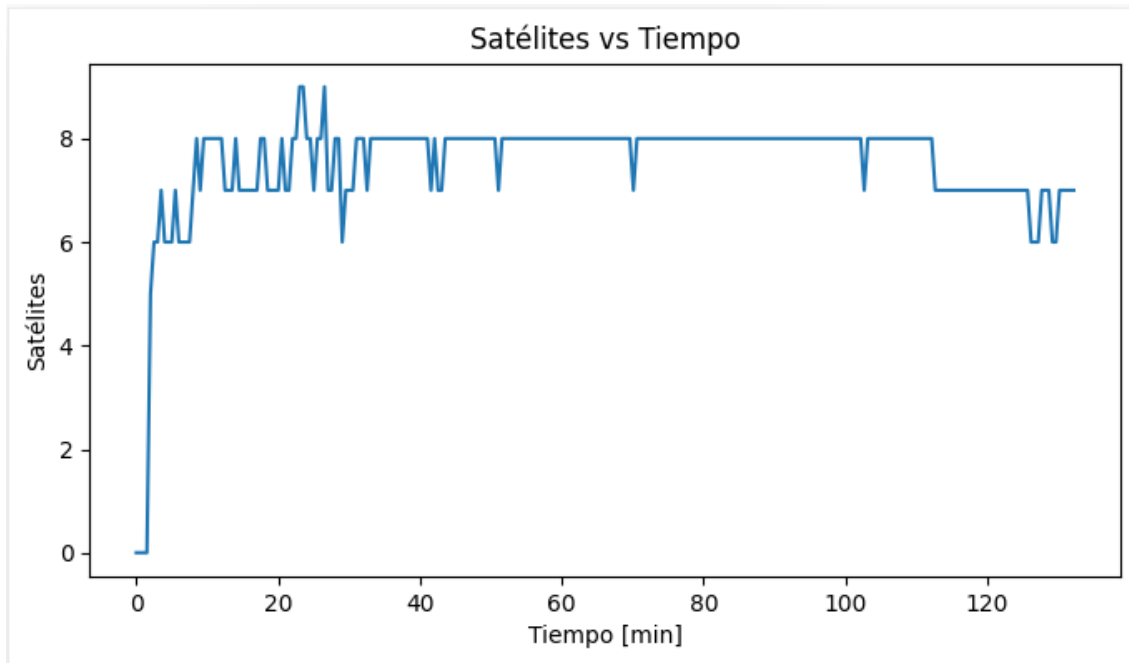
- . punto de inicio
- . trayectoria completa
- . desvíos de navegación
- . punto final
- . zonas de permanencia
- . recorridos repetidos
- . comparación entre distintos ensayos

La visualización sobre mapa constituye una herramienta muy útil para validar algoritmos de navegación, estimar errores de posicionamiento y analizar el comportamiento general del vehículo.



En este ejemplo de la figura vemos la carga de la trayectoria que corresponde al archivo PLANNING.CSV. Superponiendo esta trayectoria que es la real, con la de planificación de los waypoints podemos ver cómo se comportó el vehículo durante el transcurso de la misión.

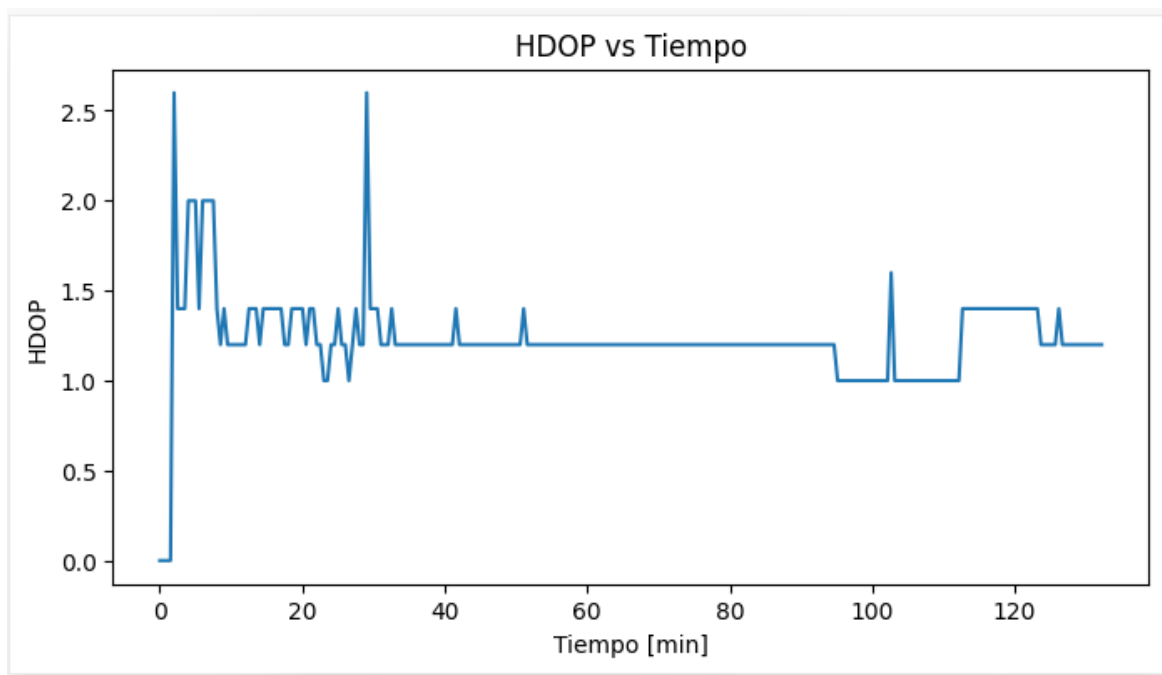
Análisis de datos registrados



Además de la visualización geográfica, los archivos CSV permiten realizar estudios estadísticos y gráficos utilizando herramientas como Excel, LibreOffice Calc o Python.

Entre los análisis más útiles se encuentran:

Calidad de recepción GPS



Tiempo vs HDOP

Permite observar la calidad geométrica de la constelación satelital durante toda la navegación.

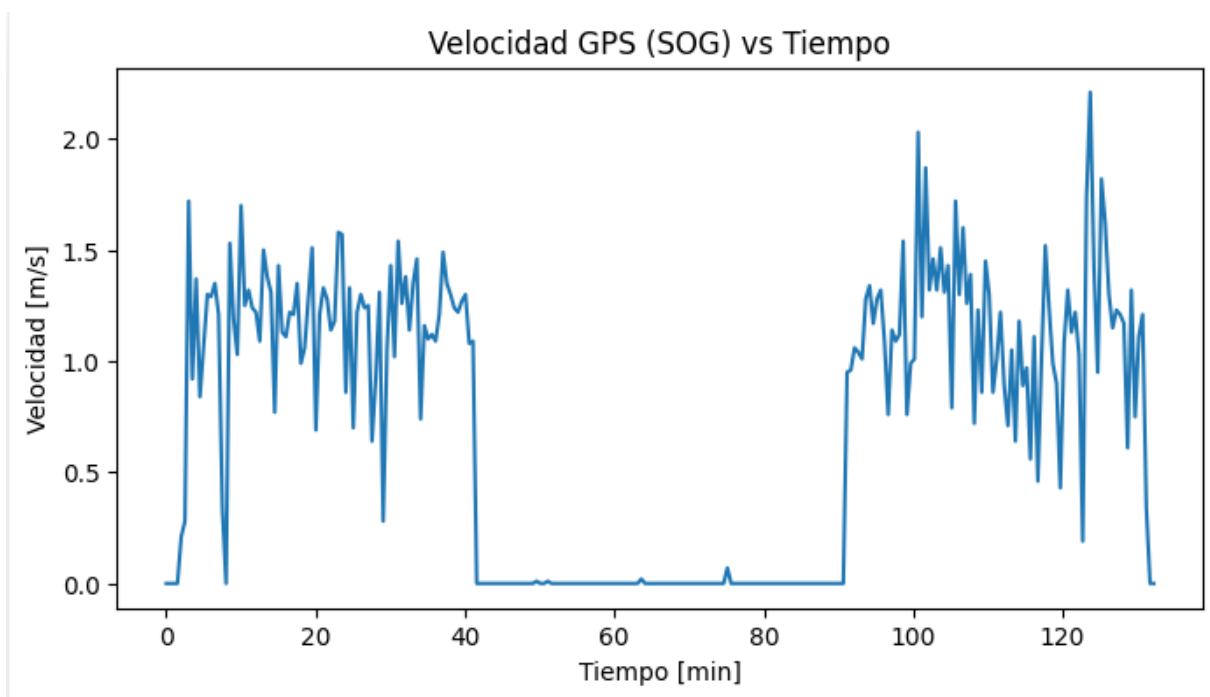
Valores bajos indican una mejor precisión de posicionamiento.

Cantidad de satélites

Tiempo vs Satélites

Permite verificar la estabilidad de recepción y detectar pérdidas temporales de señal.

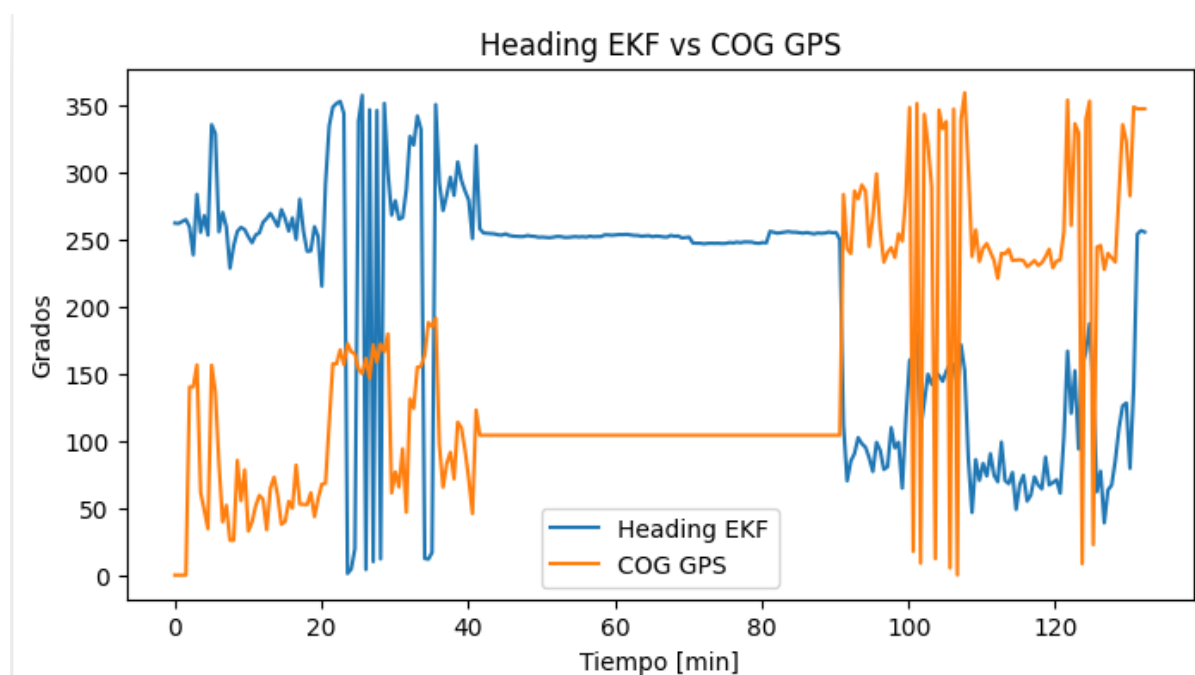
Velocidad de navegación



Tiempo vs Velocidad GPS

Permite analizar aceleraciones, desaceleraciones y comportamiento dinámico del vehículo.

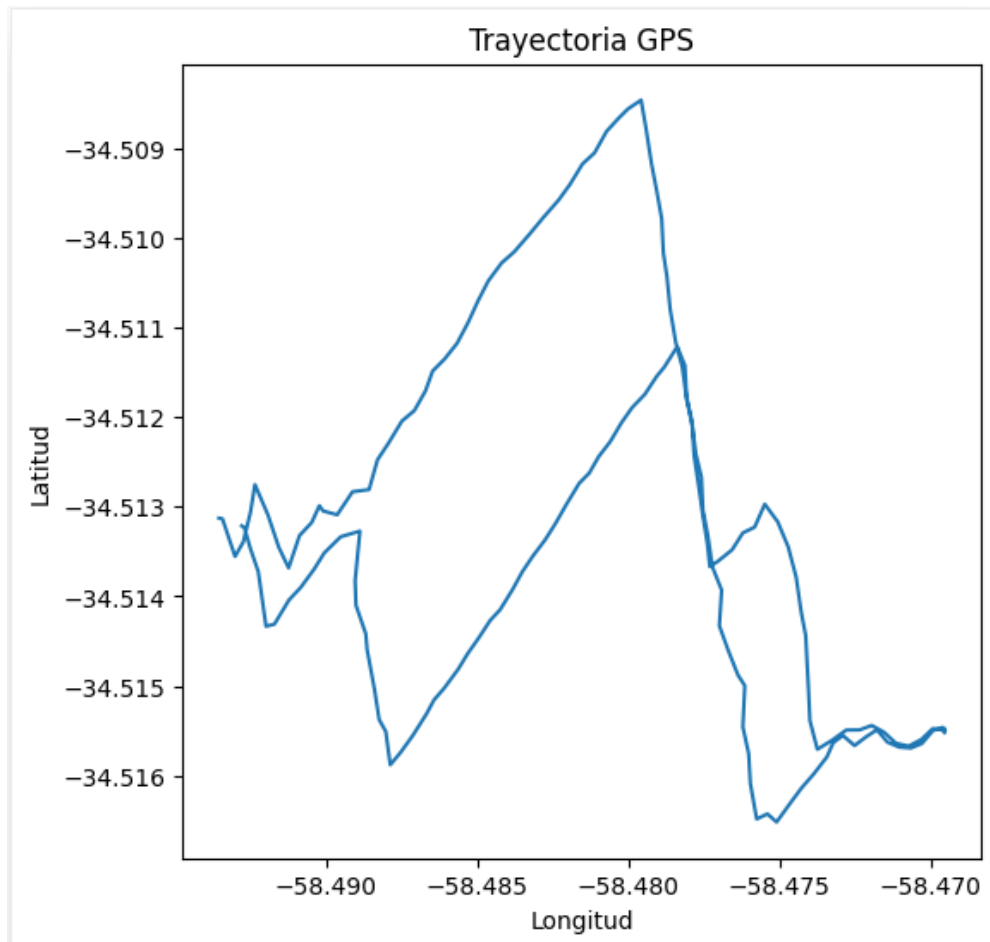
Comparación de rumbos



Rumbo EKF vs Rumbo GPS (COG)

Permite validar el desempeño del filtro EKF y detectar posibles errores en sensores inerciales o magnetométricos.

Precisión GPS



Tiempo vs EHPE

Permite evaluar la incertidumbre horizontal estimada por el receptor GPS.

Análisis de permanencia

Cuando el vehículo permanece detenido durante un período prolongado, los datos almacenados permiten cuantificar la deriva estática del GPS y estimar el error real de posicionamiento.

Esta prueba resulta particularmente útil para caracterizar el comportamiento del receptor GPS utilizado por el autopiloto antes de integrarlo completamente al sistema de navegación autónoma.

Aplicaciones prácticas

El sistema de registro de recorridos permite:

- validar algoritmos de navegación
- comparar rutas planificadas y rutas reales
- analizar errores de posicionamiento GPS
- evaluar el desempeño del EKF

- detectar problemas de sensores
- medir precisión de llegada a waypoints
- documentar pruebas de campo
- generar históricos de navegación
- construir mapas de recorridos realizados

De esta manera, la memoria microSD deja de ser únicamente un medio de almacenamiento y pasa a convertirse en una herramienta fundamental para el desarrollo, depuración y validación del sistema de autopiloto.

Conclusión

La incorporación de la memoria microSD y de la interfaz USB MSC representa una mejora significativa en las capacidades de desarrollo, diagnóstico y operación del sistema de autopiloto. La posibilidad de almacenar configuraciones, misiones, calibraciones y registros históricos permite desacoplar gran parte de la información del firmware, facilitando modificaciones y ensayos sin necesidad de recompilar el software.

La integración del sistema de registro de navegación basado en archivos CSV agrega además una herramienta de análisis extremadamente valiosa para la validación del sistema. Mediante el almacenamiento periódico de parámetros de navegación es posible reconstruir recorridos completos, visualizar trayectorias sobre mapas, analizar el desempeño del GPS y evaluar el comportamiento de los algoritmos de estimación y control implementados.

Estas capacidades convierten a la memoria microSD en un componente estratégico dentro de la arquitectura del autopiloto, proporcionando una plataforma robusta para la realización de pruebas de campo, depuración avanzada y validación experimental de futuras etapas del proyecto.